Battleship: An Agile Programming Project

Mohamed Charfi - s221586
Ahmed Aziz Ben Haj Hmida - s221551
Ghalia Bennani - s221649
Ons Riahi - s221565
Thibaud Bourgeois - s221592

April 8, 2026

Contents

1 Introduction **3**

2 User Stories **3**

3 System Design **3**
3.1 UML Class Diagram 3
3.2 Software Architecture and Design Patterns 3
3.2.1 battleship.controller 3
3.2.2 battleship.game 4
3.2.3 battleship.gui 5
3.2.4 battleship 6
3.3 Design patterns 6
3.3.1 Singleton Pattern 6
3.3.2 Observer Pattern 6
3.3.3 MVC Pattern 7

4 Project Management **7**
4.1 Work Distribution 7

5 User Manual **7**
5.1 Getting Started 7
5.2 Game Rules 7
5.3 Winning the Game 8

6 Additional Resources **9**
6.1 Screencast 9
6.2 GitHub Repository 9

7 Implemented Functionalities **9**
7.1 Bonus 1: the Game Menu and sounds 9
7.2 Bonus 2: the timed version of the game 9
7.3 Bonus 3: the rocks 9
7.4 Bonus 4: the bombs 9
7.5 Bonus 5: the game informations 9

1 Introduction

This report presents the development process and outcomes of a Battleship game project as part of the Agile Object-oriented Software Development course at DTU. Battleship is a classic strategy game in which two players place their ships on a grid and take turns trying to sink each other's ships by guessing their coordinates. The player who sinks all of their opponent's ships first wins the game.

Our team of five has implemented the game following the Agile software development methodology, focusing on user stories, iterative development, and continuous improvement. The project includes mandatory functionalities, such as implementing the game's rules, supporting multiplayer and AI modes, and providing a graphical user interface for user interaction. In addition, we have implemented four optional functionalities to enhance the game.

The purpose of this report is to provide an overview of the project's development process, including user stories, system design, project management and work distribution. Additionally, we will present a UML Class diagram, a description of the software architecture and design patterns used, and a simple user manual. The report also contains the URLs to the screencast of the running game, demonstrating all implemented functionalities, and the URL to the Git repository.

2 User Stories

`https://drive.google.com/file/d/1Nut7h3EzEWDVR2qYwvZDeEGcpfUiAXV1/view?usp=sharing`

3 System Design

3.1 UML Class Diagram

`https://drive.google.com/file/d/1EsaY6NJMRR8_5ss9z9blWZKW959Augf2/view?usp=sharing`

3.2 Software Architecture and Design Patterns

In this section, we present an overview of the system's design, focusing on the classes/interfaces and their relationships.

The system is divided into three packages: *battleship.game*, *battleship.gui* and *battleship.controller*. The *battleship.game* package contains the core game logic, while the *battleship.gui* package is responsible for the graphical user interface and the *battleship.controller* accepts input and converts it to commands for the game or GUI. The parent *battleship* package contains the main class that launches the application.

3.2.1 battleship.controller

The *battleship.controller* package includes the following classes/interfaces:

- **GameController**: This class represents the controller of the "Battleship" game. It links the game logic and the graphical user interface (GUI) and handles user input.

- **GameCreationListener**: This interface provides a method for notifying listeners when a game is created.
- **SoundManager**: This class provides functionality for playing various sound effects during the game, with the sound files being loaded from a resources folder. The class contains a static enum representing each sound effect, and methods for playing and stopping those sound effects using JavaFX "MediaPlayer" instances.

3.2.2 battleship.game

The *battleship.game* package includes the following classes:

- **AIPlayer**: The AIPlayer class represents an artificial intelligence player in the Battleship game. The class extends the Player class and adds the logic for placing ships and handling shots. The AI player can operate in random or basic mode, with the latter using a more sophisticated targeting strategy.
- **Board**: This class represents the game board in the Battleship game. It stores the number of rows and columns, as well as a matrix of Cell objects. The class provides methods to access and modify the cells on the board, as well as filling the board with ocean and rock cells.
- **Cell**: This class represents a single cell on a game board in a battleship game. It contains the coordinates of the cell, the status of the cell, and a list of observers that are notified when the status of the cell changes. The class provides methods to retrieve the coordinates and cell status, as well as to update the cell status and add observers to the list. It also includes a private method to notify the observers when the cell status changes.
- **CellStatus** (enum): This enumeration represents the possible states of a cell on the game board, such as "HIT", "MISSED", "SHIP", "OCEAN", "ALREADY_MISSED", "ALREADY_HIT", "ROCK", "ROCK_HIT".
- **Coordinates**: This class represents a pair of coordinates (column and row). The class provides methods to get the column and row values, and overrides toString() and equals().
- **Game**: This class represents a Battleship game, containing boards, players, and methods to switch players, check the current player, and generate the game elements based on game mode and settings.
- **Orientation** (enum): This enumeration represents the possible orientations of a ship on the game board: North, South, East or West. It is used in the Player class when placing ships on the board.
- **Player**: This abstract class is the base class for both human (RealPlayer) and AI (AIPlayer) players in the Battleship game. It contains a list of the player's remaining ships, the game board, and the player's ID. The class provides methods for placing ships on the board and handling shots.

- **PlayerId** (enum): This class is an enumeration that represents the two possible players in the Battleship game: PLAYER_1 and PLAYER_2. The class provides a method next() that returns the next player in the game.
- **RealPlayer**: This class represents a human player in the Battleship game. This class extends the Player class and inherits its properties and methods.
- **Ship**: This class represents a ship in the Battleship game. It stores a list of Cell objects that represent the ship, as well as the ShipType of the ship. The class provides methods to access the list of cells and check if the ship has sunk.
- **ShipType** (enum): The ShipType enumeration represents the different types of ships in the game with their sizes. The class provides methods to get the ship's size, and a string representation of the ship type.

The *RealPlayer* and *AIPlayer* classes extend the abstract *Player* class, providing specific implementations for human and AI players, respectively.

3.2.3 battleship.gui

The *battleship.gui* package includes the following classes:

- **BattleShipButton**: This class extends the JavaFX Button class and creates a custom button for the Battleship game.
- **BattleShipSubScene**: This class extends the JavaFX SubScene class and creates a custom subscene for the menu.
- **CellObserver** (interface): This is an interface that defines a method cellUpdated(). It is used for updating the cell representation in the graphical interface.
- **GameInfo**: This class is responsible for storing and managing game information.
- **GameMode** (enum): This enumeration defines the available game modes such as MULTIPLAYER, EASY, and HARD.
- **GraphicalCell**: This class extends the JavaFX Canvas class and implements the CellObserver interface. It provides a graphical representation of a cell. It manages the cell's appearance based on its status and provides methods for redrawing the cell.
- **GraphicalGame**: This class is responsible for setting up the graphical interface of the game depending on the game mode. It initializes the game by creating PlayerPane instances for each player.
- **IdGenerator**: This class is a simple utility class to generate unique IDs.
- **Info**: This class generates in-game messages for different events, such as ship placement, hit, miss, and player wins. It takes a PlayerId object in its constructor and formats the messages accordingly.
- **InfoLabel**: This class extends the JavaFX Label class to create a customized label with specific font and background style. It's used for displaying information during the game.

- **PlayerObserver**: This is an interface with a single method onRemainingShipsChanged. Every class implementing this interface can receive updates on the remaining number of ships for a player.
- **PlayerPane**: This class extends the Pane class. It provides a visual representation of the game state for both the main player and the other player. The class handles user inputs for ship placement, manages the grid display, keeps track of game information, and updates the timer if the game mode is timed.
- **SceneManager**: This class manages the game's main menu and submenus, including scenes for setting difficulty, game modes, and displaying the game rules.
- **StringsEn**: This class is a collection of static final strings used as button labels, window titles, prompts, and other in-game messages.

3.2.4 battleship

- **Battleship**: This class is the main entry point for the Battleship game application. It extends the Application class. The class sets up the main stage, initializes the game, and plays background music throughout the game.

3.3 Design patterns

In the development of the Battleship game, several design patterns were employed to ensure a modular, maintainable, and efficient code structure.

3.3.1 Singleton Pattern

The Singleton pattern was applied to the `Game` and `GraphicalGame` classes. By creating unique static private `Game/GraphicalGame` objects in the classes and making their constructors private and returning the created objects in `getInstance()` methods, we ensured that only one instance of each class is created throughout the lifetime of the application. This prevents multiple instances from conflicting with each other and helps manage global state.

```
private static Game instance = null;
private Game() {
}
public static Game getInstance() {
    if (instance == null) {
        instance = new Game();
    }
    return instance;
}
```

3.3.2 Observer Pattern

In the Battleship game implementation, the observer pattern was employed to facilitate communication between different components in the game. Three observer interfaces were utilized: `CellListener`, `PlayerListener`, and `GameCreationListener`.

- **GameCreationListener**: the SceneManager emits an event when a game mode is selected. Then the BattleShip class can listen to this event and instantiate the GameController accordingly. This way, the SceneManager remains focused on managing scenes and UI elements, while the responsibility of creating the game is delegated to a separate class.
- **CellListener**: This interface enables the `Cell` class to notify the GraphicalCell whenever the cellStatus is changed. the game logic is seperated from the graphical logic, promoting a cleaner code structure.
- **PlayerListener**: This interface facilitates communication between the `Player` class and other components of the game. This is used to observe the number of remaining ships so that the progress bar changes.

3.3.3 MVC Pattern

The model view controller pattern is used to link the game logic with the graphical interface. The class GameController sets the PlayerPane on mouseClicked, on keyPressed and on mouseReleased depending on the game mode and the game progression.

4 Project Management

4.1 Work Distribution

The work has been distributed this way:
The model part has been done by everyone.
The view part has been done by Aziz, Mohamed and Thibaud.
The AI algorithms for both modes easy and hard have been done by Ghalia and Ons.
The Controller part has been done by everyone.
The bonuses have been done by Aziz, Mohamed and Thibaud. The report has been done by everyone.

5 User Manual

5.1 Getting Started

In this Battleship game, you will face off against your opponent, strategically placing your fleet of ships and taking turns to attack each other's ships. The game features a "Rules" button that displays the rules of the game when pressed.

5.2 Game Rules

1. You and another player are facing each other, and neither can see the other's board.
2. Place your fleet of 5 ships on the board.
    - 2 rocks will appear in random positions in each of the 2 players ocean grids.
    - **Rules for placing ships:**

- For each ship, select one of the orientations: for NORTH : ↑ or **W**, for SOUTH : ↓ or **S**, for WEST ← or **A**, for EAST → or **D** and choose a cell from your ocean grid to place the ship according to that orientation.
- Do not place a ship so that any part of it overlaps with the edge of the grid, or another ship.
- Do not change the position of any ships once the game has begun.

3. You and your opponent will alternate turns, calling out one shot per turn to try to hit each other's ships.

4. On your turn, pick a target cell from the opponent's ocean grid.

   - **It's a hit!**
     - If you call out a shot location occupied by a ship on your opponent's ocean grid, your shot is a hit!
   - **It's a miss...**
     - If you call out a shot location not occupied by a ship on your opponent's ocean grid, it's a miss.
   - **It's a rock...**
     - If one player calls out a shot location occupied by a rock on the opponent's ocean grid, then the other player will be granted a bomb bonus for his next shot which when hitting a cell instead of revealing just one cell, nine cells (a 3x3 square) are revealed. This allows the other player to uncover potential ship locations more quickly.

5. Once all the cells of any of the ships are hit, the ship will sink and the progress bar of the player who sunk that ship will be incremented.

6. There are four game modes in this version:

   - **Multiplayer**
     - a classic two-player mode where players take turns attacking each other's ships.
   - **Blitz Multiplayer**
     - similar to the regular multiplayer mode, but each player has only 5 seconds to make a move. If a player doesn't make a move within the allotted time, their turn is skipped and the other player gets to play.
   - **Easy Bot**
     - a solo mode against an AI opponent with an easy difficulty level.
   - **Hard Bot**
     - similar to the Easy Bot mode, but the AI opponent has a higher difficulty level making it more challenging to defeat.

5.3 Winning the Game

The first player to sink all of their opponent's ships wins the game.

6 Additional Resources

6.1 Screencast

A video demonstration of the project can be found at the following link: `https://youtu.be/fMEB7YwzuNY`

6.2 GitHub Repository

The source code for the project is available on GitHub at the following link: `https://github.com/thibaudbrg/Agile-OO-SoftDev-Project`

7 Implemented Functionalities

7.1 Bonus 1: the Game Menu and sounds

We created a game menu so that the player can read the rules and choose the game mode before starting to play. Sound effects are implemented in the menu as well as in the game itself whenever you click on a menu button, hit a ship, win a game ... Animations are also implemented when clicking on buttons or moving from a scene to another in the menu of the game.

7.2 Bonus 2: the timed version of the game

We created a new game mode in which every player has 5 seconds to make a move. Otherwise his turn is discarded.

7.3 Bonus 3: the rocks

We added rocks in the map that spawn randomly so that the placement of ships becomes more complex.

7.4 Bonus 4: the bombs

Whenever a player shoots at a rock, the other player will have a bomb in the next turn. The bomb will hit all the neighbouring cells.

7.5 Bonus 5: the game informations

We added a section in each player's pane in which we display a custom game information dialog box so that each player can know his progression in the game (whenever he sinks a ship, which ship the player is going to place, a progression bar,...)